Decision Models for Microservices: Design Areas, Stakeholders, Use Cases, and Requirements

Stefan Haselböck[1(✉)], Rainer Weinreich[1], and Georg Buchgeher[2]

[1] Johannes Kepler University Linz, Linz, Austria
{stefan.haselboeck,rainer.weinreich}@jku.at
[2] Software Competence Center Hagenberg GmbH, Hagenberg im Mühlkreis, Austria
georg.buchgeher@scch.at

Abstract. Introducing a microservice architecture is a complex task, requiring many design decisions regarding system architecture, organizational structure, and system infrastructure. Decision models have been successfully used in other domains for design space exploration, decision making and decision documentation. In this paper, we investigate the use of decision models for microservice architecture. As a first step, we identified areas of microservice design and created decision models for some of the identified areas. We then used the created models as part of a technical action research (TAR) process with partner companies to identify important stakeholders and use cases for decision models in this context, as well as to identify requirements on decision model elements and presentation. Results indicate that practitioners perceive decision models for microservices to be useful. Challenges include the large number of interlinked knowledge areas, the need for context-specific adaptations, and the need for processes to manage the decision space over time.

Keywords: Decision models · Microservices · Technical action research (TAR)

1 Introduction

Microservice architecture (aka Microservices) is an architectural style that focuses on modularizing service-oriented software systems in such a way that services and service development teams are as independent from each other as possible [27]. While modularization has always been a means to enable independent development, microservices are also built for independent deployment, scalability, and evolution.

Shifting to a microservice architecture promises faster time-to-market of individual services, less coordination and thus less complexity through development teams that act independently, and more efficient utilization of resources by enabling the possibility to scale individual services independently from each other.

© Springer International Publishing AG 2017
A. Lopes and R. de Lemos (Eds.): ECSA 2017, LNCS 10475, pp. 155–170, 2017.
DOI: 10.1007/978-3-319-65831-5_11

However, introducing a microservice architecture is no easy task. A company undertaking such an endeavor needs to make a variety of decisions touching a wide array of development practices, technologies, and infrastructure to handle the additional challenges and complexity that accompany this architectural style. Examples of such changes are the introduction of an automatic deployment pipeline to deal with the continuous deployment of individual services, the use of a runtime infrastructure that provides isolation and scalability, the introduction of a monitoring infrastructure, and changes to the release process and testing practices, along with organizational and procedural changes (e.g., DevOps [3]). All in all, changing to a microservice architecture may touch nearly every aspect of service development, including design, infrastructure, development practice, and team organization.

To handle this complexity, we investigate the use of decision models to support the establishment of a microservice architecture. Decision models are a well-known approach for exploring the design space, making decisions, documentation, and reuse in software architecture [5,24]. Early work on design space exploration and semi-formal models for representing the design space in human computer interaction (HCI) was performed by MacLean et al. [15,16]. Later, models for representing and utilizing architectural decisions were developed as part of research on rational management [6] and knowledge management [1] in software architecture. In [24], we identified more than 50 approaches to software architecture knowledge management, many of them based on semi-formal decision models. Recent work in this area has also investigated decision models for guiding decision making in different domains, like service-oriented systems [30], cloud computing [29], and cyber-foraging systems [12].

In this paper, we first identify potential areas for microservice decision making. After creating initial decision models for some of the identified areas, we used those models in a technical action research (TAR) process with partner companies to identify important stakeholders and use cases for decision models in the context of microservice architecture. Finally, we identify requirements for the decision models themselves, such as required elements to address the identified use cases, along with specifying requirements concerning the presentation of the decision models to the identified stakeholders.

The remainder of the paper is structured as follows. Section 2 contains background information and outlines related work on decision models. We describe the research design and research questions in Sect. 3. The identified areas of microservice design, stakeholders, use cases, and requirements on microservice decision model elements and presentation are presented in Sect. 4. Section 5 summarizes the main results. Threats to validity are discussed in Sect. 6, and Sect. 7 concludes.

2 Background and Related Work

Bosch and Jansen [4,11] defined software architecture as the result of a set of architectural decisions in one of the first decision-oriented views of software architecture. Since then, design decisions have become important elements of software

architecture research, and several models, methods, and tools have been developed to represent, capture, and manage decisions and their rationales in software architecture [5,23,24]. Decisions are also a central concept in the ISO/IEC/IEEE 42010 standard on software architecture documentation [10].

Early work on decision models in human computer interaction (HCI) was conducted by MacLean et al. [15,16], who introduced QOC (Questions, Options, and Criteria) as a semiformal notation to represent the design space around an artifact. QOC focused on representing the basic concepts of design space analysis: questions, options, and criteria. The QOC notation was later adopted in research on architectural knowledge management [1] and used as a basis for decision models in software architecture [14,28].

Zdun [28] proposed decision models based on QOC for analyzing the software design space. The models are meant to be reusable for multiple design decisions. The work on QOC-based decision models for software architecture was further refined and supported by tools in the ADvISE approach [14].

Zimmermann and Miksovic [30] presented decision models for service-oriented architecture, including the possibility to reuse one or more guidance models in their SOAD meta-model; they described guidance models as reusable assets. In [29], they extended their previous work on decision models, introducing the concepts of problem-space and solution-space modelling.

Lewis et al. [12] presented decision models for cyber-foraging systems with the goal of providing guidance for the architecture and evolution of such systems. In contrast to Zimmermann et al. [29], they did not include a mechanism for capturing decisions made. Their models are comparable to the guidance models proposed by Zimmermann and Miksovic in [30].

Here, we develop decision models for a microservice architecture, specifically using as a basis a metamodel similar to that of Lewis and Zimmermann. Our metamodel includes concerns which relate to questions in the QOC notation and to requirements in other decision models, design options, and implications, which can also be used as criteria to evaluate design options. Implications relate to tradeoffs in the decision model by Lewis et al. [12], but they may not only be related to system qualities but also to organizational structures, economic implications, and infrastructure. We use this metamodel as a basis for developing and refining decision models for various important microservice topics, such as service discovery, service registration, versioning, load balancing, caching, fault tolerance, and so on (see, e.g., [9] on decision models for monitoring microservices). As part of this process, we also identified important areas of microservice design, important stakeholders for these models, potential use cases, and specific requirements for decision models in this domain.

3 Research Design

The goal of this study is to identify areas of microservice design where decision models might be used, stakeholders and use cases for decision models in this context, and requirements for elements of decision models and their presentation. We defined the following research questions to address this goal:

- **RQ1:** What are the main areas of design for a microservice-based system, and how can they be categorized?
- **RQ2:** Which are the important stakeholders of decision models for microservice architectures?
- **RQ3:** What are the intended use cases for decision models for microservice architectures?
- **RQ4:** What elements of a microservice decision model and their presentation are required to support stakeholders in their intended used cases?

3.1 Literature Review

To identify areas of design for microservice architectures and to categorize the identified areas, we reviewed the literature in the digital libraries of IEEE, ACM, and Springer. We first performed an automatic search using the following search terms: microservice*, micro-service*, and "micro service*". We searched for these terms in the titles, abstracts, and keywords of articles published in journals, conferences, and workshops.

After the automatic search, we manually analyzed the results, removing articles not relevant for this study. We then analyzed the remaining articles with main topics regarding different areas of microservice decision-making. Finally, we included several existing mapping studies on microservices [2,7,19] in our analysis.

In addition to the above literature review, we also manually reviewed two well-known books [18,27] and two blogs [13,21] on microservices to validate and refine the list of identified areas of microservice design.

3.2 TAR Study

To identify stakeholders, use cases, and requirements of decision models for microservices, we performed a technical action research (TAR) study as described by Wieringa [26]. The concept of TAR is to design artifacts and validate them under real-world conditions.

The basic structure of the TAR process is shown in Fig. 1, comprising three different cycles. In each cycle, the researcher plays a different role. In the design cycle, the researcher designs an artifact to solve problems in a research context. In the empirical cycle, the researcher validates the designed artifact and tries to answer the defined research questions. The client engineering cycle has two different aims. The client wants to solve a problem with the designed artifact, and the researcher wants to validate his or her artifact. Thus, the researcher helps the client solve their problem by applying the designed artifact. At the same time, he or she collects information to answer the research questions.

We performed the TAR study with two different companies that are currently establishing microservice architectures. The companies operate in two different domains, banking and process automation. The artifacts under study were use cases and stakeholders of decision models for microservices, as well as required elements for decision models and presentation of microservices. To collect data

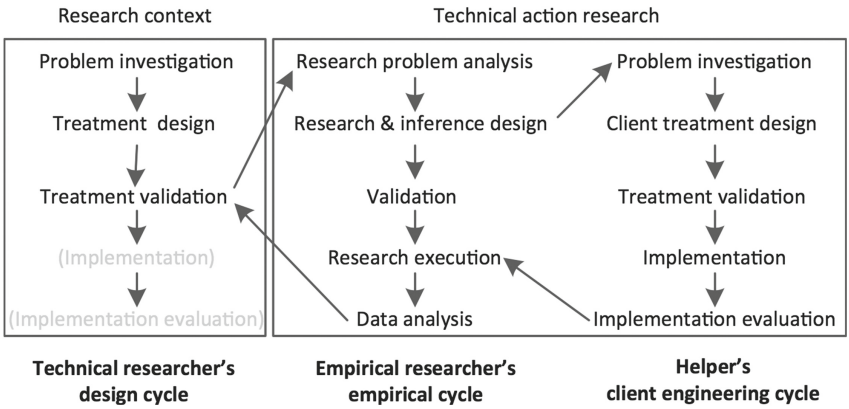

Fig. 1. The three-level structure of TAR (from [26])

for these points, we performed four client engineering cycles with the two companies. In each cycle, we participated in a design workshop with the clients, each addressing a particular area of microservice design. The workshops were part of regular design sessions at the companies that took place every two weeks as part of Scrum cycles. While we initiated participation in the workshops, we did not initiate the workshops themselves. In preparation for the workshops, we created decision models for the defined areas based on the microservice literature (the helper's role). During the workshop, the participants used the created models to discuss potential design decisions regarding the issues in a particular design area in their company's context, while we (as researchers) acted as pure observers and data collectors. At the end of each workshop, we additionally interviewed workshop participants about potential stakeholders, intended use cases, and the provided models to obtain feedback about the elements and presentation. Interviews were performed either by client staff or by the researchers. The following briefly describes the four different client engineering cycles.

- In the *first client engineering cycle*, we participated in a design workshop on service discovery at a company in the banking domain. During this cycle, the client aimed to get an overview of the topic of service discovery, define requirements for service discovery, and discuss potential design options. We provided initial models for service discovery, service registration, and load balancing, which the workshop participants could use in their discussion, but we did not interfere otherwise.
- The *second engineering cycle* was performed with the same client as was the first cycle. The aim of this cycle was to reflect design decisions already made for service discovery. We used the revised metamodel created during the first client engineering cycle.
- In the *third client engineering cycle*, we participated in a workshop on resilience with the same company. In this cycle, the client had the same aim as in the first engineering cycle. We used a decision model for resilience, which

was based on the refined metamodel from the previous client engineering cycles.
- The *fourth client engineering cycle* was done with a company in the process automation domain. We participated in a meeting with the client on microservice monitoring. The client aimed to get an overview of the topic of monitoring and of available design options, to define requirements for microservice monitoring in the company's context, and to get some information about existing monitoring technologies. We used a set of decision models for microservice monitoring [9] in this workshop.

The main means of data collection were observation and unstructured interviews [22]. We took field notes during the workshops, which we extended with information collected in interviews. For analysis of the collected data, we used the constant comparison method [22] to collect and refine the data in each client engineering cycle. We performed data analysis in two steps. First, we coded the field notes using a provisional "start list" [17] with predefined categories and several subcategories. We then analyzed the coded field notes, finally creating a field memo [22] including the identified stakeholders, use cases, and necessary changes to the elements or presentation. Furthermore, based on the field memo, we redesigned the decision metamodel and the presentation of the model (i.e., the notation of the model) for the next client engineering cycle.

4 Results

In the following section, we present the results that address the research questions presented in Sect. 3, starting with the identified design areas, stakeholders, and use cases (RQ1–RQ3). Section 4.4 presents the requirements for decision models and their elements (RQ4) that emerged during the client engineering cycles in the TAR process.

4.1 RQ1: Design Areas

Figure 2 overviews the design areas that were identified in the literature review. We briefly describe each main design area.

Integration: Microservice architectures comprise potentially thousands of independently developed and operated services, which need to be integrated with each other. Integration of services can occur at different levels, such as the user interface level, the service level, or the data level.

Modularization: Microservice architecture is often primarily seen as a specific, modular approach that aims to design microservices as independently from each other as possible. This may lead to services that provide their own user interfaces and data management facilities. Concepts like domain-driven design can decouple services based on their functionalities. Furthermore, service granularity and size must be considered.

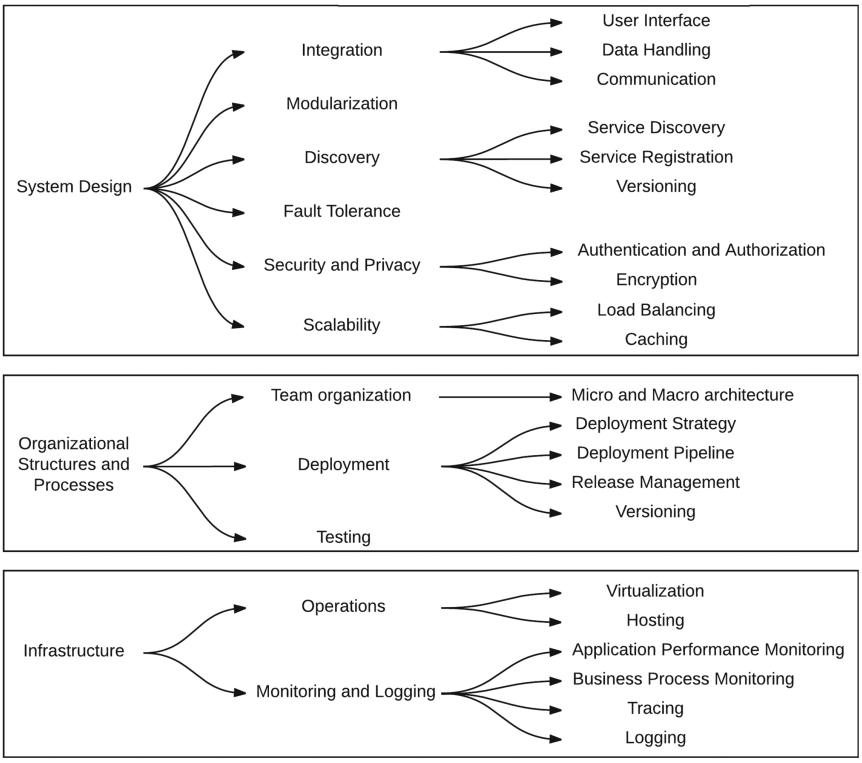

Fig. 2. Areas of microservice design

Discovery: Since microservices are typically spread across different hosts, mechanisms for service registration and discovery are needed to ensure that services can find each other. This also includes mechanisms to find specific versions of services.

Fault Tolerance: To ensure that a local failure of a microservice does not propagate through the whole system and eventually lead to system breakdown, services must be designed to be tolerant against failures at runtime. Furthermore, the infrastructure must provide mechanisms to deal with failing service instances. If required, new service instances should be started automatically.

Security and Privacy: Since microservices provide public interfaces and use remote communication spanning network boundaries, security and privacy must be considered, including subareas like authentication, authorization, and encryption.

Scalability: Scalability is often a central reason for companies to migrate towards a microservice architecture, because it enables independent and selective scaling of single services. Scalability can be achieved on different levels, for instance by replicating services, splitting the functionality for which a service is

responsible, or splitting the data for which a service is responsible. To cope with independently scaling services, load balancing and caching mechanisms must also be considered.

Team organization: Design also goes hand-in-hand with team organization. In a microservice architecture, teams operate as independently from each other as possible. Instead of functional teams organized around technical boundaries, teams are mostly cross-functional and organized around business capabilities. A team may develop several microservices, but the development of any single microservice is always the responsibility of a single team. Teams responsible for a microservice may also independently decide the programming language, technology stack, and development process to use.

Deployment: Independent deployment is a central motivation for using a microservice architecture. Independent deployment of a potentially large number of services requires a strategy to bring the services into production with reasonable effort, e.g., by automating the deployment pipeline with continuous deployment. In cases where complete releases require deployment at once, perhaps because of legal restrictions, release management strategies are needed.

Testing: As with deployment, testing a potentially large number of independently developed microservices with frequent releases requires automation. Services should be tested as part of a continuous delivery pipeline. Integration tests should be performed at the end of a continuous deployment pipeline for each service individually, which requires the establishment of test zones.

Monitoring and Logging: Monitoring microservice-based systems is challenging, because of the high number of independent services, which may be located on different hosts. This requires support for the automatic collection, distribution, and combination of the monitoring data from the different services. Monitoring should include application performance, business processes, tracing, and logging.

Operations: A dedicated runtime infrastructure is needed to operate microservice instances, handle the independent scaling of services, support fault tolerance, and support continuous deployment. This may include the use of virtualization and cloud infrastructures.

Some of the above areas are related to each other, such as service discovery and service registration, so making decisions in one design area might require decisions in another design area. In addition, any design area might be related to multiple, other design areas. For example, service versioning may be related to service discovery, because service discovery must handle different versions of a microservice. Also, versioning may be related to service deployment, since different versions of a service must be deployed.

The identified design areas can be further mapped to three main areas of microservice system design, organizational structures and processes, and microservice infrastructure (again see Fig. 2). This highlights that, when establishing a microservice architecture, decisions must be made not only at the level of system design, but also concerning team organization and system infrastructure.

4.2 RQ2: Stakeholders

The second research question aims to identify important stakeholders of decision models for microservices. To answer this research question, we analyzed the roles of the workshop participants in the client engineering cycles and also asked about their roles in interviews after the engineering cycles. We identified six main stakeholders for decision models.

Software Architect: At least one software architect participated in each client engineering cycle. Software architects were responsible either for the architecture of the entire microservice-based software system (i.e., the macro-architecture) or for the architecture of single microservices as part of development teams (i.e., the micro-architecture). Software architects' main interests are to use decision models to explore new design areas, provide decision guidance for other stakeholders, efficiently document design decisions, and review design decisions already made.

Developer: Developers only participated in the fourth client engineering cycle. Typically, developers are involved only in selected areas as part of a cross-functional team. Their main interest is in using decision models to make, document, and review decisions as part of the team.

Application Engineer: This stakeholder was identified during the fourth client engineering cycle. Application engineers are typically responsible for realizing customer projects based on a software platform or product line of an industrial solutions provider. As such, their main interest lies in understanding important platform decisions, along with integrating client solutions with the client's hardware and software infrastructure.

Operations Engineer: This stakeholder was present in three different client engineering cycles, which is no surprise since development and operations usually work together in the context of microservices. They were interested mostly in the design spaces provided for deployment, infrastructural issues like the runtime infrastructure used for load distribution and monitoring, and the available technology options.

Quality Assurer: Stakeholders for quality assurance were present in all client engineering cycles. They were usually part of service development and cross-functional teams and mainly interested in changes to test processes and fault handling.

Manager: This stakeholder was identified during the first and second client engineering cycles. Some workshop participants mentioned that they have to provide support for decision-making that is the responsibility of people at higher management levels. Currently, this is done with the time-consuming method of PowerPoint presentations. They expect to use the structured way of presenting design options and implications together with their rationales to streamline this process.

While the different stakeholders were partly interested in different decision models, typically related to their areas of responsibility and interest, it also

became obvious that decision models would typically be used in a team setting, where stakeholders with expertise or interest in a particular topic were present. In addition, other people required for discussion (e.g., experts on specific infrastructure) were integrated on-demand through conference calls.

4.3 RQ3: Use Cases

RQ3 concerns the identification of intended use cases of decision models for microservices. To answer this question, we identified how the decision models were used in the client engineering cycles. In the interviews after the workshops, we also asked study participants to identify additional potential use cases of the presented models. As a result, we identified the following main use cases.

Design Space Exploration: In the first use case, decision models are used to explore a potential design space and to discuss potential design options. Workshop participants used the provided decision models to overview available design options, rationales for selecting particular options, and implications of each option for system qualities and organizational structures and processes. Stakeholders participating in these discussions included software architects, developers, application engineers, quality assurance managers, and operations people. For this kind of discussion, global decision models are typically used. In our case, most of the performed client engineering cycles fell into this category.

Decision Guidance and Documentation: In this use case, decision models are used as a framework to make decisions and efficiently document decisions. The team responsible for a decision uses the models to choose one or several design options. The models provide addressed concerns with, implications of, and the components needed for each possible decision. Potential stakeholders for this use case are cross-functional teams comprising developers, architects, quality assurance, and operations. The participants in our client engineering cycles outlined two use cases in this area: (1) the creation and continuous adaptation of decision models to the local design context of the company, what we termed local or company-specific decision models, which might be used to capture cross-project knowledge in the company and guide the design process as outlined above; and (2) stakeholders at the companies also argued that decision models might be suitable for discussing important decisions with upper management, since it enables a presentation of the rationale for each element and the implications of selecting a particular option in a structured and compact way.

Design Review: In this use case, decision models are used to reflect decisions that have already been made. Therefore, existing decision models created by the decision maker, other teams, or external experts are used. In the client engineering cycles in this case, participants mentioned two intended purposes: (1) to reflect a decision made by the same team, whether architectural or cross-functional; and (2) to reflect decisions made by another team. A concrete example mentioned was the architectural team reflecting decisions made by cross-functional teams. The second client engineering cycle, described in Sect. 3.2, was

performed on the basis of this use case. The client's architectural team reflected their decisions about service discovery in the context of a particular infrastructure for service discovery. The researchers created the decision model used.

Requirements Elicitation: This use case was identified in the fourth client engineering cycle of the TAR process. The workshop participants used the provided decision models to determine the requirements of their monitoring infrastructure when discussing different monitoring options.

Evaluation of Technology Options: As expressed in the third and fourth client engineering cycles, decision models may be used to select and evaluate existing technology options for a specific area of design. This requires information about the implemented functionality and the product and runtime costs of different technology options.

4.4 RQ4: Decision Model Elements and Presentation

The following requirements concerning model elements and presentation emerged when using and refining the provided decision models in the TAR process outlined in Sect. 3.2.

Scope: In workshops that aimed at exploring a specific design space, participants added several additional elements to the model, such as concerns, implications, or design options. Some of the added elements were company-specific, but others were also valid in other contexts. Thus, different decision models are needed for different application scopes. So, we introduced the concept of context-dependent (local) and context-independent (global) decision models. Since some of the added elements were also valid in other domains, we further added a feedback loop to continuously incorporate changes to local models into more general, global models, analogous to the feedback loop from application engineering to domain engineering in software product lines described in [20].

Components: Workshop participants were often interested in the system components needed to implement a design option. As a result, we identified a need to include system components as first-level elements in decision models, which can then be linked to concerns and design options.

Technology Options: In two client engineering cycles, one important aim of the client was to identify and evaluate different technology options for implementing a specific design option. Therefore, in the decision models, we need additional elements to model technology options and link them with specific design options.

Reading Direction: To increase the models' comprehensibility, some workshop participants mentioned the need for a clear reading direction. They expressed the need to start at two different points when discussing the design space: either starting with a design option to determine which concerns were addressed or starting with concerns to determine the design options best addressing those concerns. The model's presentation must support both scenarios.

Structuring and Connecting Design Areas: Since areas of microservice design are often related, the model must provide a mechanism to define such relations between different areas. Furthermore, there are many different design areas, requiring a mechanism to structure the whole design space, for example by defining namespaces for decision models and by refining decision models into submodels.

Focusing on Important Elements: In the workshops, we noticed that the presentation of the decision models must be as simple as possible while still providing the full level of detail when needed. Otherwise, a model's users easily become overwhelmed by its complexity. Also, some elements of a particular type turned out to have different levels of importance than others. This became especially evident when showing implications, which we included in the model to show the results of selecting a particular design option. One strategy might be to classify elements according to their levels of importance, showing less important elements only when requested.

5 Discussion

Identified stakeholders were not only the software architects that would be expected in a microservice context, although software architects were often the driving force in discussions in the performed client engineering cycles and were involved in all design areas that discussed in the workshops. Other stakeholders, like developers, operation engineers, and quality assurance, were typically involved only in selected areas, according to their specific roles and interests. However, all stakeholders considered the decision models to be useful, in part because they provided a common ground of understanding and terminology and a means for structuring discussion on a topic.

The most relevant use cases in our client engineering cycles were design space exploration and reviews of already made design decisions. Also, the teams were heavily driven in part by existing microservice technology stacks. Therefore, the decision models were also used to evaluate existing technology options and how they implemented specific design options. In the fourth client engineering cycle, in particular, the company used the decision models to determine requirements for their monitoring infrastructure. The decision models not only enabled a higher level of completeness but also served as a means for weighing the importance of requirements in light of how they addressed design options. Additional use cases like decision guidance, documenting decisions and their rationales, and refining models were identified as equally important in the long term. Finally, participants identified the aspect of sharing as important, including adaptations of local decision models in concrete usage scenarios and the continuous refinement of shared global models based on these context-specific changes.

The decision models we created for the client engineering workshops (e.g., the microservice monitoring models presented in [9]) include the main elements of QOC models (see Sect. 2). In addition, our models included further elements, such as components and technology options, to support the evaluation and

selection of technology options and to obtain an impression of what would be required to implement a design option.

The presentation of a single model should mainly provide an overview of the design space, with details shown as needed. Additionally, decision models for a large number of identified design areas must be structured with, for example, namespaces and submodels. We plan further investigation of different forms of model presentation to stakeholders in our ongoing TAR study with both current and new business partners.

6 Threats to Validity

The presented results of the literature review and TAR study may be influenced by several factors.

In our literature review, we did not intend to obtain a complete list of areas of design for microservices. Instead, we wanted an overview of important areas of design that must be considered when introducing microservice architectures and that we could potentially support with decision models. We tried to mitigate threats related to the representativeness of the list of identified design areas by performing an automatic search of the main digital libraries for software engineering using a quite general search string (see Sect. 3.1), followed by manual selection of relevant studies from the results of the automated search. Afterwards, we extended and refined the resulting list of studies with results of mapping studies, augmented with important books and blog posts on this topic.

In our TAR study, we used the data collection methods of observation and unstructured interviews. Seaman [22] mentioned a number of threats to consider for both means of data collection.

For observations, the participants should not feel observed, and the observer should not interrupt in any way. To address these issues, we explained our approach and aims to participants before each session, making it clear that we would not interfere with the workshop in any way. During the workshops, we acted as pure observers and data collectors. We only answered questions regarding the meaning of the elements in the discussed decision models.

A potential threat regarding interviews is that interviewees may answer questions in a socially desirable way [25]. To mitigate this threat, interviews should not be performed by the researchers, if possible. In our case, the interviews were partly performed by client staff and partly by the researchers. In cases where we performed the interviews, we made clear that there were no right or wrong answers, as suggested in [22].

For analysis of collected data during the observations and interviews, we used the constant comparison method [22]. Gasson [8] mentioned threats that must be addressed when using this method, namely confirmability, dependability, internal consistency, and transferability. To ensure confirmability, each researcher independently coded the field notes and discussed the findings. To address dependability, we followed the TAR study process, with observations and interviews used as data collection methods and the constant comparison method used for

data analysis. To ensure internal consistency, we derived the data from the field memos, which were created based on the independent coding of the field notes. Transferability was addressed by using the artifact under study in several client engineering cycles. In each cycle, the context of design space, use case, participants, or client changed.

Still, our study faces some limitations that may affect the external validity of the findings, most importantly the limited number of clients and client engineering cycles. We tried to mitigate this threat by working with different clients from different domains, as well as by working with different teams at one of the clients. Furthermore, in the client engineering cycles we used decision models for different areas of design.

7 Conclusion

In this work, we identified potential areas of microservice design through a literature review, created decision models based on existing work, decision models in other domains, and microservice literature, and then used the created models in a technical action research process with industrial partners to identify stakeholders, use cases, and requirements for decision models for microservices.

Our results showed that there is a wide range of potential stakeholders in this domain, from software architects and operations personnel to management. Stakeholders outlined a set of use cases, including requirements elicitation, design space exploration, decision guidance, technology evaluation, decision documentation, and decision presentation. Quite interestingly, the provided decision models worked well during workshops undertaken as part of the TAR client engineering cycles, with the models seen not as necessary evils but rather being naturally integrated as useful elements for structuring discussions and supporting the outlined use cases.

Using the models in a real-world context also helped identify specific requirements for the models themselves and their presentation to stakeholders, including the necessity of adapting decision models to a local context, the need for a feedback loop to refactor changes made to local models to models with more global scope, the need to structure a large design space, and the need to model relationships among many design models. The basic elements of the decision models remained broadly similar to what was proposed in QOC; we needed only to add elements like technology options and components to address specific use cases. Finally, we need to give more thought to the presentation of models to support specific use cases, with emphasis placed on presenting only the least amount of information necessary for a potential use case in order to disturb the process at hand as little as possible.

In future work, we plan to perform further workshops as part of our ongoing TAR study. Through these workshops, we aim to refine and validate decision models for the identified areas of microservice design with our partner companies, scientific partners, and possibly with other companies. We are currently working on a tool to support this process of continuous communication and refinement.

Acknowledgement. The research reported in this paper has been supported by the Austrian Ministry for Transport, Innovation and Technology, the Federal Ministry of Science, Research and Economy, and the Province of Upper Austria in the frame of the COMET center SCCH.

References

1. Ali Babar, M., Dingsøyr, T., Lago, P., Van Vliet, H. (eds.): Software Architecture Knowledge Management: Theory and Practice. Springer, Heidelberg (2009)
2. Alshuqayran, N., Ali, N., Evans, R.: A systematic mapping study in microservice architecture. In: 2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA), pp. 44–51, November 2016
3. Bass, L., Weber, I., Zhu, L.: DevOps: A Software Architect's Perspective. Addison-Wesley Professional, Boston (2015)
4. Bosch, J.: Software architecture: the next step. In: Oquendo, F., Warboys, B.C., Morrison, R. (eds.) EWSA 2004. LNCS, vol. 3047, pp. 194–199. Springer, Heidelberg (2004). doi:10.1007/978-3-540-24769-2_14
5. Capilla, R., Jansen, A., Tang, A., Avgeriou, P., Babar, M.A.: 10 years of software architecture knowledge management: practice and future. J. Syst. Softw. **116**, 191–205 (2015)
6. Dutoit, A.H., McCall, R., Mistrík, I., Paech, B. (eds.): Rationale Management in Software Engineering. Springer, Heidelberg (2006). doi:10.1007/978-3-540-30998-7. http://link.springer.com/10.1007/978-3-540-30998-7
7. Francesco, P.D., Malavolta, I., Lago, P.: Research on architecting microservices: trends, focus, and potential for industrial adoption. In: Proceedings of the 14th International Conference on Software Architecture (ICSA), Gothenburg, Sweden, April 2017
8. Gasson, S.: Rigor in grounded theory research: an interpretive perspective on generating theory from qualitative field studies. In: Whitman, M.E., Woszczynski, A.B. (eds.) The Handbook of Information Systems Research, pp. 79–102. Idea Group, Hershey (2004)
9. Haselböck, S., Weinreich, R.: Decision guidance models for microservice monitoring. In: IEEE International Workshop on Architecting with MicroServices (AMS), International Conference on Software Architecture Workshops (ICSAW 2017), Gothenburg, Sweden, April 2017
10. ISO/IEC/IEEE 42010–2011(E): Systems and software engineering - Architecture description. IEEE (2011). doi:10.1109/IEEESTD.2011.6129467
11. Jansen, A., Bosch, J.: Software architecture as a set of architectural design decisions. In: 5th Working IEEE/IFIP Conference on Software Architecture (WICSA 2005), pp. 109–120 (2005)
12. Lewis, G.A., Lago, P., Avgeriou, P.: A decision model for cyber-foraging systems. In: 13th Working IEEE/IFIP Conference on Software Architecture (WICSA), pp. 51–60, April 2016
13. Lewis, J., Fowler, M.: Microservices, Mar 2014. https://martinfowler.com/articles/microservices.html
14. Lytra, I., Tran, H., Zdun, U.: Supporting consistency between architectural design decisions and component models through reusable architectural knowledge transformations. In: Drira, K. (ed.) ECSA 2013. LNCS, vol. 7957, pp. 224–239. Springer, Heidelberg (2013). doi:10.1007/978-3-642-39031-9_20

15. MacLean, A., McKerlie, D.: Design space analysis and use-representations. In: Carroll, J.M. (ed.) Scenario-Based Design: Envisioning Work and Technology in System Development. Wiley, New York (1995)
16. MacLean, A., Young, R.M., Bellotti, V.M.E., Moran, T.P.: Questions, options, and criteria elements of design space analysis. Hum. Comput. Interact. **6**(3), 201–250 (1991). http://dx.doi.org/10.1207/s15327051hci0603&4_2
17. Miles, M.B., Huberman, A.M.: Qualitative Data Analysis: An Expanded Sourcebook. Sage, Thousand Oaks (1994)
18. Newman, S.: Building Microservices. O'Reilly Media Inc., Sebastopol (2015). http://shop.oreilly.com/product/0636920033158.do
19. Pahl, C., Jamshidi, P.: Microservices: a systematic mapping study. In: Proceedings of the 6th International Conference on Cloud Computing and Services Science - Volume 1: CLOSER, pp. 137–146 (2016)
20. Pohl, K., Böckle, G., van Der Linden, F.J.: Software Product Line Engineering: Foundations, Principles and Techniques. Springer Science & Business Media, Heidelberg (2005)
21. Richardson, C.: What are microservices?. http://microservices.io
22. Seaman, C.B.: Qualitative methods. In: Shull, F., Singer, J., Sjøberg, D.I.K. (eds.) Guide to Advanced Empirical Software Engineering, pp. 35–62. Springer, London (2008). doi:10.1007/978-1-84800-044-5_2
23. Tofan, D., Galster, M., Avgeriou, P., Schuitema, W.: Past and future of software architectural decisions a systematic mapping study. Inf. Softw. Technol. **56**(8), 850–872 (2014). http://www.sciencedirect.com/science/article/pii/S0950584914000706
24. Weinreich, R., Groher, I.: Software architecture knowledge management approaches and their support for knowledge management activities: a systematic literature review. Inf. Softw. Technol. **80**, 265–286 (2016). http://www.sciencedirect.com/science/article/pii/S0950584916301707
25. Wieringa, R., Moralı, A.: Technical action research as a validation method in information systems design science. In: Peffers, K., Rothenberger, M., Kuechler, B. (eds.) DESRIST 2012. LNCS, vol. 7286, pp. 220–238. Springer, Heidelberg (2012). doi:10.1007/978-3-642-29863-9_17
26. Wieringa, R.J.: Design Science Methodology for Information Systems and Software Engineering. Springer, Heidelberg (2014). http://www.springer.com/de/book/9783662438381
27. Wolff, E.: Microservices: Flexible Software Architecture. Addison-Wesley Professional, New Jersey (2016)
28. Zdun, U.: Systematic pattern selection using pattern language grammars and design space analysis. Softw. Pract. Exp. **37**(9), 983–1016 (2007). http://onlinelibrary.wiley.com/doi/10.1002/spe.799/abstract
29. Zimmermann, O., Wegmann, L., Koziolek, H., Goldschmidt, T.: Architectural decision guidance across projects - problem space modeling, decision backlog management and cloud computing knowledge. In: 2015 12th Working IEEE/IFIP Conference on Software Architecture (WICSA), pp. 85–94 (2015)
30. Zimmermann, O., Miksovic, C.: Decisions required vs. decisions made connecting enterprise architects and solution architects via guidance models. In: Aligning Enterprise, System, and Software Architectures, p. 176 (2012)